

Java Regular Expressions (Regex)

1. What is Regex?

Regular Expressions (Regex) are a sequence of characters defining a **search pattern**. They are commonly used for **validations**, **searching**, **text manipulation**, and **data extraction**.

Java provides the **java.util.regex** package to handle regular expressions.

2. Java Regex API: Pattern and Matcher Classes

Java provides two main classes to work with regex:

1. **Pattern** → Represents a compiled regex pattern.
2. **Matcher** → Performs operations like searching, matching, and replacing.

Example Code

```
import java.util.regex.*;

public class RegexExample {

    public static void main(String[] args) {

        // Define the regex pattern

        String regex = "\\d{3}"; // Matches exactly 3 digits

        Pattern pattern = Pattern.compile(regex);
```

```
// Input string to check

String input = "The number is 123 and code is 456.";


// Create matcher object

Matcher matcher = pattern.matcher(input);

// Find matches

while (matcher.find()) {

    System.out.println("Matched: " + matcher.group());

}

}
```

Output:

```
Matched: 123
```

```
Matched: 456
```

3. Regex Metacharacters

Metacharacters are **special characters** with a specific meaning in regex.

Metacharacter	Description	Example
.	Matches any character except newline (<code>\n</code>)	<code>a.c</code> → Matches <code>abc</code> , <code>axc</code> , but not <code>ac</code>
<code>\d</code>	Matches any digit (0-9)	<code>\d+</code> → Matches <code>123</code> , <code>89</code>
<code>\D</code>	Matches non-digit characters	<code>\D+</code> → Matches <code>hello</code>
<code>\w</code>	Matches word characters (<code>a-z</code> , <code>A-Z</code> , <code>0-9</code> , <code>_</code>)	<code>\w+</code> → Matches <code>hello123</code>
<code>\W</code>	Matches non-word characters	<code>\W+</code> → Matches <code>@@@</code>
<code>\s</code>	Matches whitespace characters (space, tab, newline)	<code>\s+</code> → Matches
<code>\S</code>	Matches non-whitespace characters	<code>\S+</code> → Matches <code>hello</code>
<code>^</code>	Matches the beginning of a string	<code>^Hello</code> → Matches <code>"Hello world"</code> but not <code>"Hi Hello"</code>

\$	Matches the end of a string	world\$ → Matches "Hello world" but not "world Hello"
----	------------------------------------	---

Example Code for Metacharacters

```
import java.util.regex.*;

public class MetacharactersExample {

    public static void main(String[] args) {

        String regex = "\\d\\D"; // Matches a digit followed by a
non-digit

        Pattern pattern = Pattern.compile(regex);

        String input = "7a 8b 99";

        Matcher matcher = pattern.matcher(input);

        while (matcher.find()) {

            System.out.println("Matched: " + matcher.group());

        }

    }

}
```

Output:

Matched: 7a

Matched: 8b

4. Regex Quantifiers

Quantifiers define **how many times a character or group appears** in the text.

Quantifier	Description	Example
*	Matches 0 or more occurrences	<code>a*b</code> → Matches <code>b</code> , <code>ab</code> , <code>aaab</code>
+	Matches 1 or more occurrences	<code>a+b</code> → Matches <code>ab</code> , <code>aaab</code> but not <code>b</code>
?	Matches 0 or 1 occurrence	<code>a?b</code> → Matches <code>b</code> , <code>ab</code>
<code>{n}</code>	Matches exactly n occurrences	<code>\d{3}</code> → Matches <code>123</code> , <code>456</code> , but not <code>12</code>
<code>{n, }</code>	Matches at least n occurrences	<code>\d{2, }</code> → Matches <code>12</code> , <code>12345</code>

<code>{n,m}</code>	Matches between n and m occurrences	<code>\d{2,4}</code> → Matches 12, 1234, but not 1
--------------------	--	--

Example Code for Quantifiers

```
import java.util.regex.*;

public class QuantifierExample {

    public static void main(String[] args) {

        String regex = "\\d{2,4}"; // Matches 2 to 4 digits

        Pattern pattern = Pattern.compile(regex);

        String input = "12 123 12345 1";

        Matcher matcher = pattern.matcher(input);

        while (matcher.find()) {

            System.out.println("Matched: " + matcher.group());

        }

    }

}
```

```
}  
  
}
```

Output:

```
Matched: 12  
  
Matched: 123  
  
Matched: 1234
```

5. Common Regex Patterns

5.1 Email Validation

Regex:

```
^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z]{2,}$
```

Explanation:

Each part of the regex serves a specific purpose in validating an email format.

1 ^ (Caret)

- Indicates the start of the string.
- Ensures that matching begins from the very first character.

2 [a-zA-Z0-9_+ -]+

- Matches the username (before @).
- Allows:
 - Lowercase letters (a-z)
 - Uppercase letters (A-Z)
 - Numbers (0-9)
 - Underscore (_)
 - Dot (.)
 - Plus (+)
 - Hyphen (-)
- The + means one or more occurrences of these characters.

♦ Example Matches:

✓ "john.doe"

✓ "user_123"

✓ "test.email+filter"

3 @ (At Symbol)

- This literally matches the "@" character.
- It separates the username from the domain name.

♦ Example Matches:

✓ "john.doe@"

✓ "user_123@"

4 [a-zA-Z0-9-]+

- Matches the domain name (before .).

- **Allows:**
 - Lowercase letters (**a-z**)
 - Uppercase letters (**A-Z**)
 - Numbers (**0-9**)
 - Hyphen (**-**) (but not at the start or end)
- The **+** means one or more occurrences.

◆ **Example Matches:**

✓ "gmail"

✓ "yahoo-mail"

✓ "company123"

5 \. (Escaped Dot)

- The **.** (dot) is a special character in regex, so we use **\.** to match a literal dot.
- Separates the domain name from the domain extension.

◆ **Example Matches:**

✓ "gmail.com"

✓ "yahoo.co.uk"

6 [a-zA-Z]{2,}

- Matches the top-level domain (TLD) like **.com**, **.org**, **.net**, etc.
- **{2,}** means at least 2 characters, allowing:
 - **.com**
 - **.org**
 - **.edu**
 - **.co.uk** (partial match, but full validation would need further regex).

♦ **Example Matches:**

✓ "com"

✓ "org"

✓ "io"

7 \$ (Dollar Sign)

- Indicates the end of the string.
- Ensures no extra characters appear after the email address.

✓ **Example Valid Email Matches:**

john.doe@gmail.com

user_123@yahoo.com

test.email+filter@domain.co.uk

my-email@company123.org

✗ **Example Invalid Email Matches:**

Invalid Email	Why It's Invalid?
user@.com	Domain name missing

@gmail.com	Username missing
user@gmail..com	Double dots not allowed
user@-domain.com	Hyphen cannot start domain
user@domain.c	TLD must be at least 2 characters
user@domain#com	Invalid character #

Summary

Regex Part	Meaning
^	Start of the string
[a-zA-Z0-9_+-.]+	Username: Letters, digits, _, ., +, -
@	Literal @ symbol

<code>[a-zA-Z0-9-]+</code>	Domain name (letters, digits, -)
<code>\.</code>	Literal . (dot)
<code>[a-zA-Z]{2,}</code>	TLD (at least 2 letters)
<code>\$</code>	End of the string

Example Code:

```
import java.util.regex.*;

public class EmailValidation {

    public static void main(String[] args) {

        String regex =
"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.\.[a-zA-Z]{2,}$";

        Pattern pattern = Pattern.compile(regex);

        String[] emails = {"test@example.com", "invalid@",
"user@domain"};
```

```
for (String email : emails) {  
  
    Matcher matcher = pattern.matcher(email);  
  
    System.out.println(email + " is valid: " +  
matcher.matches());  
  
}  
  
}  
  
}
```

5.2 Password Strength Validation

A **strong password** should:

- Be **at least 8 characters** long.
- Contain at least **one uppercase letter**.
- Contain at least **one lowercase letter**.
- Contain at least **one digit**.
- Contain at least **one special character**.

Regex:

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$
```

Explanation:

This regex enforces **strong password rules**, ensuring a mix of uppercase, lowercase, numbers, and special characters.

1 ^ (Caret)

- Indicates the **start** of the string.
 - Ensures that the pattern applies from the **beginning** of the password.
-

2 (?=.*[A-Z])

- Ensures at least one uppercase letter (**A-Z**).
- **?=** is a **lookahead assertion**, which means:
 - The password **must contain** at least one uppercase letter, but it **does not consume** the character.

♦ Example Matches:

✓ "Pass123!" (✓ Contains "P")

✗ "password123!" (✗ No uppercase letter)

3 (?=.*[a-z])

- Ensures at least one lowercase letter (**a-z**).
- Works similarly to the uppercase rule.

♦ Example Matches:

✓ "Pass123!" (✓ Contains "a")

✗ "PASSWORD123!" (✗ No lowercase letter)

4 (?=.*\d)

- Ensures at least one digit (**0-9**).

♦ **Example Matches:**

✓ "Pass123!" (✓ Contains "1")

✗ "Password!" (✗ No digit)

5 (? = . * [@ \$! % * ? &])

- Ensures at least one special character (@\$!%*?&).
- Only allows specific special characters.

♦ **Example Matches:**

✓ "Pass123!" (✓ Contains "!")

✗ "Password123" (✗ No special character)

6 [A - Z a - z \ d @ \$! % * ? &] { 8 , }

- Defines the valid characters:
 - A-Za-z → Uppercase & lowercase letters
 - \d → Digits (0-9)
 - @\$!%*?& → Allowed special characters
- {8,} ensures at least 8 characters in total.

♦ **Example Matches:**

✓ "Pa\$\$w0rd!" (✓ 9 characters)

✗ "Pa\$\$1" (✗ Less than 8 characters)

7 \$ (Dollar Sign)

- Indicates the end of the string.

- Ensures that the **entire password** matches the criteria.

✓ Example Valid Passwords:

Strong@123

MyP@ssw0rd

Pa\$\$w0rd!

✗ Example Invalid Passwords:

Invalid Password	Why It's Invalid?
password	✗ No uppercase, number, or special character
PASSWORD123	✗ No lowercase, no special character
Pass123	✗ Less than 8 characters, no special character
Pa\$\$word	✗ No number

Summary

Regex Part	Meaning
<code>^</code>	Start of string
<code>(?=.*[A-Z])</code>	At least one uppercase letter (<code>A-Z</code>)
<code>(?=.*[a-z])</code>	At least one lowercase letter (<code>a-z</code>)
<code>(?=.*\d)</code>	At least one digit (<code>0-9</code>)
<code>(?=.*[@\$!%*?&])</code>	At least one special character (<code>@\$!%*?&</code>)
<code>[A-Za-z\d@\$!%*?&]{8,}</code>	Minimum 8 characters , only allows letters, numbers, special characters
<code>\$</code>	End of string

5.3 Phone Number Validation

Regex:

```
^\+?[0-9]{1,3}?[-. ]?[0-9]{10}$
```

Explanation:

This regex is used to validate phone numbers, supporting:

- Optional country codes (+1, +91, etc.).
- Optional separators (-, ., or a space).
- A **10-digit** phone number.

1 ^ (Caret)

- **Indicates the start** of the string.
- Ensures that the phone number is matched from the **beginning**.

2 \+?

- \+ → Matches an optional "+" at the start (for country codes).
- ? → Makes "+" **optional**, meaning numbers **can** have a country code or not.

♦ Example Matches:

✓ +91-9876543210 (✓ Includes +91)

✓ 9876543210 (✓ No country code, still valid)

✗ ++9876543210 (✗ Multiple + signs)

3 `[0-9]{1,3}?`

- `[0-9]` → Matches **digits (0-9)**.
- `{1,3}` → Allows **1 to 3 digits** (for country codes like `1`, `91`, `123`).
- `?` (Lazy quantifier) → Ensures it matches **as few digits as possible** while still fulfilling the pattern.

♦ Example Matches:

✓ `+1`

✓ `+91`

✓ `+123`

✗ `+9876` (✗ More than 3 digits)

4 `[-. ]?`

- `[-. ]` → Matches an optional **separator** (`-`, `.`, or space).
- `?` → Makes the separator **optional** (it can be present or absent).

♦ Example Matches:

✓ `+91-9876543210` (✓ Uses `-` as separator)

✓ `+91.9876543210` (✓ Uses `.` as separator)

✓ `+91 9876543210` (✓ Uses space as separator)

✓ `+919876543210` (✓ No separator)

5 `[0-9]{10}`

- `[0-9]` → Matches **digits (0-9)**.

- `{10}` → Ensures exactly **10 digits** (for standard mobile numbers).

♦ Example Matches:

✓ 9876543210 (✓ 10-digit number)

✗ 987654321 (✗ Only 9 digits)

✗ 98765432101 (✗ More than 10 digits)

6 \$ (Dollar Sign)

- Indicates the **end** of the string.
- Ensures the phone number **fully** follows the pattern.

✓ Example Valid Phone Numbers:

+91-9876543210

+1 9876543210

+123.9876543210

9876543210

✗ Example Invalid Phone Numbers:

Invalid Phone Number	Why It's Invalid?
++91-9876543210	✗ Extra + sign

+9876-9876543210	✗ Country code too long (more than 3 digits)
987654321	✗ Only 9 digits (not a valid phone number)
98765432101	✗ More than 10 digits

Summary

Regex Part	Meaning
^	Start of string
\+?	Optional + for country codes
[0-9]{1,3}?	1 to 3 digits for country code (lazy match)
[-.]?	Optional separator (-, ., or space)
[0-9]{10}	Exactly 10 digits for phone number

\$	End of string
----	---------------

6. Java Regex Methods: `find()`, `matches()`, `replaceAll()`

Method	Description
<code>find()</code>	Searches for pattern in text. Returns <code>true</code> if found.
<code>matches()</code>	Returns <code>true</code> if entire string matches the pattern.
<code>replaceAll()</code>	Replaces all occurrences of the pattern.

Example

```
import java.util.regex.*;

public class RegexMethodsExample {

    public static void main(String[] args) {

        String text = "Java is amazing. I love Java.";

        String regex = "Java";
```

```
Pattern pattern = Pattern.compile(regex);

Matcher matcher = pattern.matcher(text);

// Using find()

while (matcher.find()) {

    System.out.println("Found: " + matcher.group());

}

// Using matches()

System.out.println("Matches entire string: " +
text.matches(regex));

// Using replaceAll()

String replacedText = text.replaceAll(regex, "Python");

System.out.println("Replaced Text: " + replacedText);

}

}
```

Output:

Found: Java

Found: Java

```
Matches entire string: false
```

```
Replaced Text: Python is amazing. I love Python.
```

7. Conclusion

- `Pattern` and `Matcher` classes handle regex in Java.
- **Metacharacters** help define patterns.
- **Quantifiers** specify how many times a pattern should appear.
- `find()`, `matches()`, and `replaceAll()` are key methods.

Here are **more in-depth examples** of Java **Regular Expressions (Regex)** covering **validation, searching, replacing, and practical use cases**.

1. More on Metacharacters

Metacharacters help define **patterns** in regex. Let's look at some **real-world examples**.

Example: Extract all words from a sentence

```
import java.util.regex.*;

public class ExtractWords {

    public static void main(String[] args) {
```



```
String text = "Hello! This is a regex example: Java123,
Regex@2024.";

String regex = "\\b\\w+\\b"; // Match words (word boundary
\\b, word characters \\w+)

Pattern pattern = Pattern.compile(regex);

Matcher matcher = pattern.matcher(text);

while (matcher.find()) {

    System.out.println("Word: " + matcher.group());

}

}
```

Output:

```
Word: Hello

Word: This

Word: is
```

Word: a

Word: regex

Word: example

Word: Java123

Word: Regex

2. More on Quantifiers

Example: Validate an Indian Phone Number

```
import java.util.regex.*;

public class PhoneNumberValidation {

    public static void main(String[] args) {

        String regex = "[6-9]\\d{9}$"; // Must start with 6-9 and
        have 10 digits

        Pattern pattern = Pattern.compile(regex);

        String[] phoneNumbers = {"9876543210", "1234567890",
        "99999abcde", "8989898989"};
```

```
        for (String number : phoneNumbers) {  
            System.out.println(number + " is valid: " +  
pattern.matcher(number).matches());  
        }  
    }  
}
```

Output:

```
9876543210 is valid: true  
1234567890 is valid: false  
99999abcde is valid: false  
8989898989 is valid: true
```

3. Email Validation (Detailed)

Rules for a valid email:

- Starts with **alphanumeric characters** (a-z, A-Z, 0-9).
- May contain **dot (.)** or **underscore (_)**.
- Must contain **@domain.com**.

Regex:

```
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
```

Example Code:

```
import java.util.regex.*;

public class EmailValidation {

    public static void main(String[] args) {

        String regex =
"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$";

        Pattern pattern = Pattern.compile(regex);

        String[] emails = {"test@example.com",
"user.name@domain.com", "invalid@", "user@.com",
"valid_123@domain.org"};

        for (String email : emails) {

            System.out.println(email + " is valid: " +
pattern.matcher(email).matches());

        }

    }

}
```

```
}  
  
}
```

Output:

```
test@example.com is valid: true  
  
user.name@domain.com is valid: true  
  
invalid@ is valid: false  
  
user@.com is valid: false  
  
valid_123@domain.org is valid: true
```

4. Password Strength Validation

A **strong password** should:

- Be **at least 8 characters long**.
- Contain **at least one uppercase letter**.
- Contain **at least one lowercase letter**.
- Contain **at least one digit**.
- Contain **at least one special character** (@, \$, !, %, *, ?, &).

Regex:

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$
```

Example Code

```
import java.util.regex.*;

public class PasswordValidation {

    public static void main(String[] args) {

        String regex =
"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$";

        Pattern pattern = Pattern.compile(regex);

        String[] passwords = {"StrongPass1!", "weakpass", "Short1@",
"NoDigit@", "Valid_123@pass"};

        for (String password : passwords) {

            System.out.println(password + " is strong: " +
pattern.matcher(password).matches());
        }
    }
}
```

```
    }  
  
    }  
  
}
```

Output:

```
StrongPass1! is strong: true  
  
weakpass is strong: false  
  
Short1@ is strong: false  
  
NoDigit@ is strong: false  
  
Valid_123@pass is strong: true
```

5. Extracting Dates from a String

Let's extract all **dates** in `dd/mm/yyyy` format from a text.

Regex:

```
\\b\\d{2}/\\d{2}/\\d{4}\\b
```

Example Code

```
import java.util.regex.*;

public class ExtractDates {

    public static void main(String[] args) {

        String text = "Important dates: 12/05/2023, 15/08/2024, and  
29/02/2020.";

        String regex = "\\b\\d{2}/\\d{2}/\\d{4}\\b";

        Pattern pattern = Pattern.compile(regex);

        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {

            System.out.println("Date found: " + matcher.group());

        }

    }

}
```


Output:

```
Date found: 12/05/2023  
  
Date found: 15/08/2024  
  
Date found: 29/02/2020
```

6. Replacing Text using `replaceAll()`

We can use **regex** to **replace unwanted words or format text**.

Example: Replace multiple spaces with a single space

```
import java.util.regex.*;  
  
public class ReplaceMultipleSpaces {  
    public static void main(String[] args) {  
        String text = "This    is an    example    with multiple  
spaces.";   
  
        String replacedText = text.replaceAll("\\s+", " "); //  
Replace multiple spaces with one space
```

```
        System.out.println("Original: " + text);

        System.out.println("Formatted: " + replacedText);

    }

}
```

Output:

```
Original: This    is an    example    with multiple spaces.

Formatted: This is an example with multiple spaces.
```

7. Extracting Hashtags from a Tweet

Regex:

```
#\\w+
```

Example Code

```
import java.util.regex.*;

public class ExtractHashtags {

    public static void main(String[] args) {
```

```
String tweet = "Learning #Java #Regex is fun! #Coding  
#Development";  
  
String regex = "#\\w+";  
  
Pattern pattern = Pattern.compile(regex);  
  
Matcher matcher = pattern.matcher(tweet);  
  
while (matcher.find()) {  
    System.out.println("Hashtag: " + matcher.group());  
}  
  
}
```

Output:

```
Hashtag: #Java  
  
Hashtag: #Regex  
  
Hashtag: #Coding  
  
Hashtag: #Development
```

8. Extracting Domain Name from URLs

Regex:

```
https?:/(www\\.)?([a-zA-Z0-9.-]+)
```

Example Code

```
import java.util.regex.*;

public class ExtractDomains {

    public static void main(String[] args) {

        String text = "Websites: https://www.google.com,
http://example.org, and https://github.com";

        String regex = "https?:/(www\\.)?([a-zA-Z0-9.-]+)";

        Pattern pattern = Pattern.compile(regex);

        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {

            System.out.println("Domain: " + matcher.group(2));
        }
    }
}
```

```
    }  
  
    }  
  
}
```

Output:

```
Domain: google.com  
  
Domain: example.org  
  
Domain: github.com
```

Conclusion

- **Pattern and Matcher** are essential for regex in Java.
- Regex can be used for **validation, searching, replacing, and extracting data**.
- **Metacharacters & Quantifiers** control how regex patterns behave.